

Unit 5: Parameter Uncertainties

Computer Modelling

1 Aims

- Build a map of the likelihood of our supernova sample as a function of the parameters of our model.
- Write a Metropolis-Hastings class to analyze your data.
- Make histograms of the results.

2 Preparation

Make a new directory for this unit, and copy the Pantheon data file and your code from unit 4 into it. You can delete the testing and plotting code you made for that unit if you wish.

3 Background

3.1 Parameter spaces

In unit 4 we built a log-likelihood, which was a function of three parameters: $\log L(\Omega_m, \Omega_\Lambda, H_0)$. This function tells us how well those parameters fit the data.

We often visualize sets of parameters like this as a *parameter space* - each axis (dimension) of the space represents one parameter, and a point in the space represents a set of values for those parameters. So our three parameters from unit 4 would correspond to a 3D space, where Ω_m is the x-axis, Ω_Λ is the y-axis, and H_0 is the z-axis. The likelihood function is then a function of the position in this space.

The θ vector that we used in unit 4 and passed to our likelihood function represents a vector position in parameter space. In our case $\theta = \{\Omega_m, \Omega_\Lambda, H_0\}$ is a point in the 3D space.

3.2 Multi-dimensional likelihoods

We have seen how our likelihood from unit 4 was a function of three variables, the parameters of our model. Previously we worked out the single best-fitting set of values for our parameters, but because there is noise in our model, we cannot say that those are the only possible correct values. Instead, there is a distribution of possible values for these parameters, some very likely (like the best-fit) and some very unlikely, far away from the best fit. We want to characterise the full distribution of possible values for our parameters.

We can do this in a few ways. The simplest is a full brute-force likelihood evaluation on a grid, in which

we build a multidimensional array of likelihood values for all possible combinations of the parameters. This is only feasible for a small number of parameters.

3.2.1 Likelihood grid

To generate this likelihood “grid” we would choose a minimum and maximum value for each of our three parameters, and then make an array of parameters linearly spaced between those. The likelihood grid is then a 3D array:

$$G_{ijk} = \log L(p_0^i, p_1^j, p_2^k)$$

where L is the likelihood function, and p_0^i is the i th value of the first parameter Ω_m , and so on.

For example, we might choose:

$$p_0 \equiv \Omega_m = \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1\}$$

Then $p_0^0 = 0.2$, $p_0^1 = 0.3$, and so on. We would do the same for the other two parameters, and then evaluate the likelihood at each point in the grid.

3.2.2 Marginalization

To visualize this grid, we normally *marginalize* over one or more parameters, which essentially means integrating over them. This is a way to ask what the likelihood is for a subset of the parameters, regardless of the rest. For the first parameter, for example, this looks like:

$$P_{jk} = \sum_i \exp(G_{ijk} - G_{ijk}^{\max})$$

Where we have used the exponential because we want to integrate the likelihoods, not the log-likelihoods. This is a two-dimensional array, and we can plot it as a heat map or set of contours. We subtract off the maximum value to avoid numerical *underflow* where all the values are too tiny to represent.

The array P_{jk} represents the probability of the Ω_Λ and H_0 parameters taking the pair of values p_1^j, p_2^k respectively, regardless of the value of Ω_m . We can plot it as a heat map, like this:

Similarly we can build G_i by summing over two sets of indices, to get the likelihood of the first parameter regardless of the other two. That will give us a probability histogram of the first Ω_m parameter, for example, which we can plot as a line graph.

3.3 Monte Carlo Markov Chains

3.3.1 Overview

The grid method works well when we have a small number of parameters, but if we want, e.g. 20 values of each parameters, and have N parameters, then the total number of likelihood evaluations is 20^N , which grows very large very quickly. It can easily make it infeasible to run an analysis.

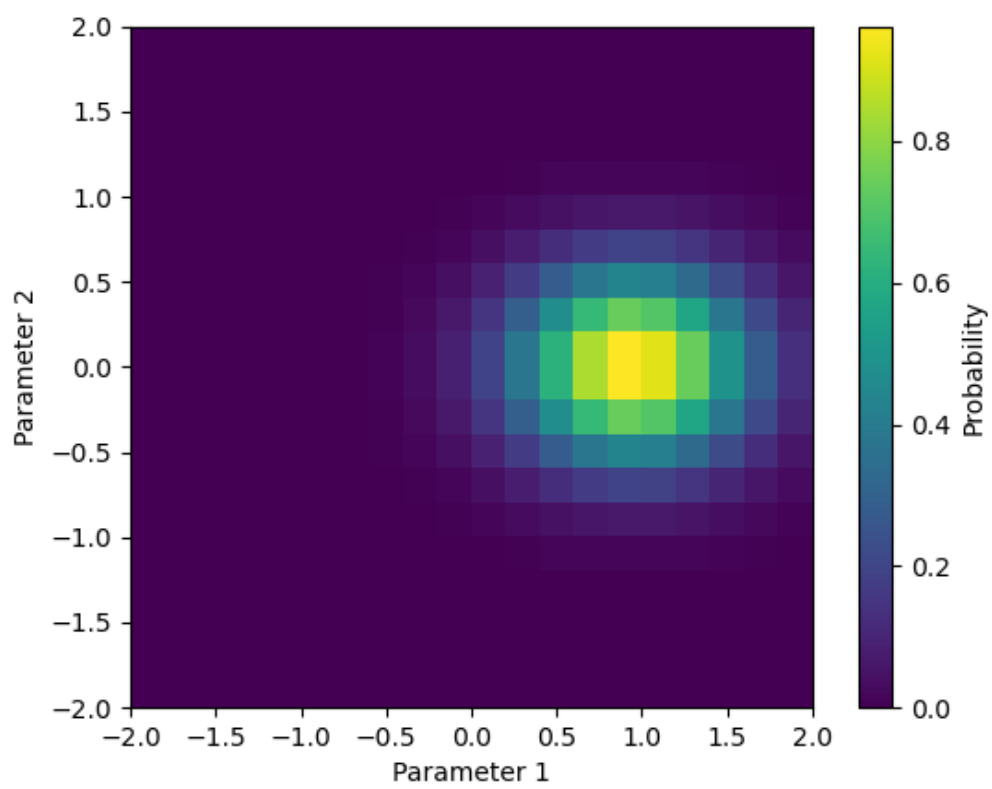


Figure 1: Heat Map

A more sophisticated class of methods called Monte Carlo Markov Chains (MCMC) are used instead in these contexts. These methods use an algorithm to “explore” the likelihood space, jumping around possible sets of parameter values using a specific rule about how to move around. It generates a sequence of parameter values that (once you run for long enough) have the correct distribution of possible values - that is, a histogram of the values in the sequence would look like the marginalized probabilities we discussed above.

3.3.2 The Metropolis Algorithm

The simplest MCMC algorithm is called the Metropolis algorithm. It works like this:

1. Start at iteration $i = 0$ at any point p_0 in the parameter space, and calculate the log-likelihood there, $\log L_0$.
2. Pick a new point p' near the current one by moving a small amount in a random direction (see “Proposal” below).
3. Evaluate the log-likelihood $\log L'$ at the new point.
4. If the $\log L' > \log L_i$ then move to the new point, setting $p_{i+1} = p'$.
5. If $L' < L_i$, then compute a random number u uniformly between 0 and 1. If $\log u < \log L' - \log L_i$, then move to the new point, setting $p_{i+1} = p'$. Otherwise, stay at the old point, setting $p_{i+1} = p_i$.
6. Record the point p_{i+1} as the current point, and go back to step 2 increasing i by one.

Steps 2-6 are repeated many times, generating a sequence of points in the parameter space. You need tens of thousands of samples for a problem like the one here.

3.3.3 Proposal

The process of picking a new point in step 2 is called the *proposal*. A good proposal makes the algorithm run very quickly; a bad one makes it run slowly. A good proposal is one that jumps around the peak of the parameter space quickly, but doesn’t jump too far away from it.

A common proposal is to move a random amount in each parameter, drawn from a Gaussian distribution with a small standard deviation, i.e.

$$\begin{pmatrix} \Omega_m \\ \Omega_\Lambda \\ H_0 \end{pmatrix} \rightarrow \begin{pmatrix} \Omega_m \\ \Omega_\Lambda \\ H_0 \end{pmatrix} + \begin{pmatrix} \sigma_{\Omega_m} \cdot \mathcal{N}_1 \\ \sigma_{\Omega_\Lambda} \cdot \mathcal{N}_2 \\ \sigma_{H_0} \cdot \mathcal{N}_3 \end{pmatrix}$$

where the $\mathcal{N}$ are independent random numbers drawn from a Gaussian distribution with mean 0 and standard deviation 1, and σ_{Ω_m} , etc., are the *step sizes* of the proposal, which you can select to make the algorithm run well.

Reasonable values for the σ values for this likelihood are: $\sigma_{\Omega_m} = 0.07$, $\sigma_{\Omega_\Lambda} = 0.1$, $\sigma_{H_0} = 0.3$.

3.3.4 Metropolis Burn-in

The Metropolis algorithm has a “burn-in” period, where the chain is still exploring the parameter space and hasn’t yet settled into the correct distribution. This is usually the first few hundred steps, and you should discard these steps when analysing the chain. You can tell when the burn-in period is over by looking at the log-likelihood values - if they are still climbing up, instead of wandering around a fixed value, then the chain is still burning in. For subsequent analysis, you should only use the steps after the burn-in period.

3.3.5 Metropolis Analysis

After running the Metropolis algorithm for a while, you can plot the probability distribution of the parameters by making a histogram of the values you recorded. This works in 1D or 2D. In 3D you can make a scatter plot and use the colour to represent the third parameter.

4 Tasks

4.1 Likelihood grid

- Make a three-dimensional likelihood grid of the full model from unit 4, as described above. While you're getting things working, just use 10 sample points in each dimension.
- Make the three 2D likelihood grids by marginalizing each parameter in turn, and the three 1D likelihoods by marginalizing over two parameters.
- Plot the 2D results using the `imshow` matplotlib function (beware that the x and y axes are swapped in this function, the first axis of the array is plotted in the y direction). Plot the 1D results as simple curves.
- Experiment with the ranges of the parameters - if the range in a parameter is too wide then the likelihood will just look like a spike in that axis. If it's too narrow then the likelihood will be very broad and flat. You want to choose a range that shows the interesting structure of the likelihood. Note in particular that the range in H_0 is quite narrow. Note your choices in your results document.
- Once you've made these choices, increase the number of sample points per dimension to as high as you can while still getting results in a few minutes. Record how long the likelihood calculations take in your results document. You can either do this manually or within python.
- Note your final chosen ranges and sample number in your results document, and save the corresponding 2D and 1D plots there too.

4.2 Metropolis

Write a class that runs a Metropolis analysis. You should make it generic so that you can use it for any likelihood function.

First you should design your class, thinking about the methods and attributes you will need. Write a short plan and discuss it with a TA or lecturer before you start coding.

Once your plan is approved by a TA, write the class, and use it to analyze the supernova sample. Collect plots and statistics as described above in the section called "Metropolis Analysis" in your results document. You will need to explore to see how many samples are needed to get smooth histograms.

Write a few sentences in your results document comparing the results of the Metropolis analysis to the likelihood grid.

5 Submitting

Submit all the python files that are needed to run your code to Learn (the cosmology class, the likelihood class, and metropolis if you put that in a new file), and a PDF of your results document. If you use the fallback cosmology class then do submit a copy of that too.

5.1 Marking

You will be marked out of 20 for this unit, and the submission is worth 10% of the final course grade. The marks are broken down as follows:

- 5 marks for the results document.
- 10 marks for code functionality, including sensible use of numpy and OOP.
- 5 marks for code quality and style, following the conventions we discussed in the Style mini-lecture. The slides for this are on Learn.